

이상 분석 탐지 시스템

SPARROW SAST에서 비정상적으로 오래 걸리는 정적 분석을 머신러닝으로 예측·탐지하고 자동으로 알리는 시스템

임영석 (Yeongseok Lim) · SP 개발 1팀, 파수(Fasoo) · 10주 프로젝트

요약

SPARROW는 파수의 정적 분석 보안 점검(SAST) 제품이다. 같은 프로젝트인데도 분석이 비정상적으로 오래 걸리는 경우가 있었지만, 이를 정상 분석과 구분할 방법이 없어 개발자가 작업이 끝날 때까지 직접 기다려야 했다. 약 700개의 GitHub 프로젝트를 분석해 메타데이터와 소요 시간을 수집하고, 28개 피처로 XGBoost 회귀 모델을 학습해 예상 분석 시간을 예측했다. 모델의 결정계수(R^2)는 평균 0.90 수준이며, 예측값의 $\pm 20\%$ 범위를 벗어나는 분석을 자동으로 탐지해 Discord로 알린다.

0.90

평균 R^2 (결정계수)

0.96

최고 R^2

~700

GitHub 저장소

28

피처

5

지원 언어

1. 배경과 문제

SPARROW는 파수의 정적 분석 보안 점검(SAST) 제품이다. 개발자가 소스코드를 제출하면 SPARROW가 정적 분석을 수행해 보안·품질 이슈를 찾아낸다.

대부분의 분석은 빠르게 끝나지만, 같은 프로젝트인데도 뚜렷한 이유 없이 분석이 비정상적으로 오래 걸리는 경우가 있었다. 정상 분석과 멈춘 듯 오래 걸리는 분석을 구분할 기준이 없었기 때문에, 개발자는 작업이 끝날 때까지 직접 기다리며 지켜봐야 했다.

표 1. 동일한 소스코드의 분석 소요 시간. 입력은 같지만 소요 시간이 비정상적으로 늘어난 분석이 존재했다.

분석 대상	분석 시작	분석 종료	소요 시간
소스코드 A	2025-06-26 13:46:58	2025-06-26 13:52:13	5분 15초
소스코드 A	2025-06-09 08:01:56	2025-06-09 13:04:56	▲ 5시간 03분
소스코드 B	2025-06-02 09:58:38	2025-06-02 09:59:51	1분 13초
소스코드 B	2025-06-02 09:02:23	2025-06-02 12:17:23	▲ 3시간 15분

목표. 프로젝트의 특성으로부터 예상 분석 시간을 학습하고, 예상 범위를 크게 벗어나는 분석을 자동으로 탐지해 알리는 시스템을 만든다.

2. 접근 방법과 시스템

데이터 수집·저장·분석·모델 개발·서비스 연동까지 전 과정을 단독으로 담당했다. 전체 파이프라인은 다음과 같다.

GitHub 저장소 수집 → 정적 분석 (소요 시간 + 메타데이터) → S3 (소스) · PostgreSQL (메타데이터) → XGBoost 학습 → 예상 시간 예측 · 편차 탐지 → Discord 알림

기술 스택

Python · pandas · NumPy · scikit-learn · XGBoost · joblib · PostgreSQL · AWS S3 · GitHub Actions · SPARROW SDK · Discord Webhook

3. 데이터 수집

SPARROW가 지원하는 언어 전반에 걸쳐 약 700개의 GitHub 공개 저장소를 수집했다. 용량 편향을 막기 위해 0-50MB 30개, 50-100MB 30개, 100-200MB 40개로 구간을 고르게 잡았다.

각 저장소에 정적 분석을 실행하고 분석 소요 시간과 함께 구조적 메타데이터를 기록했다. 언어별로 코드 용량(MB)·줄 수·순환 복잡도 합(CCN)·파일 수를 수집하고, 전체 합계와 파생 비율(MB당 줄 수, 줄당 복잡도, MB당 파일 수, 언어 개수)까지 더해 **총 28개 피쳐**를 구성했다.

표 2. 수집·가공한 피쳐 구성 (총 28개).

구분	피쳐
언어별 (5개 언어)	mb · line · ccn_sum · file_count
전체 합계	total_line · total_mb · total_ccn · total_file_count · lang_count
파생 비율	line_per_mb · ccn_per_line · file_per_mb

대상 언어: Java · C / C++ · Python · JavaScript / TypeScript · C#. 단일 언어와 다중 언어(확장자 조합) 프로젝트를 모두 포함했다.

4. 영향 요인 분석

각 요인을 분리해 보기 위해 변수를 통제하며 분석 소요 시간과의 관계를 확인했다.

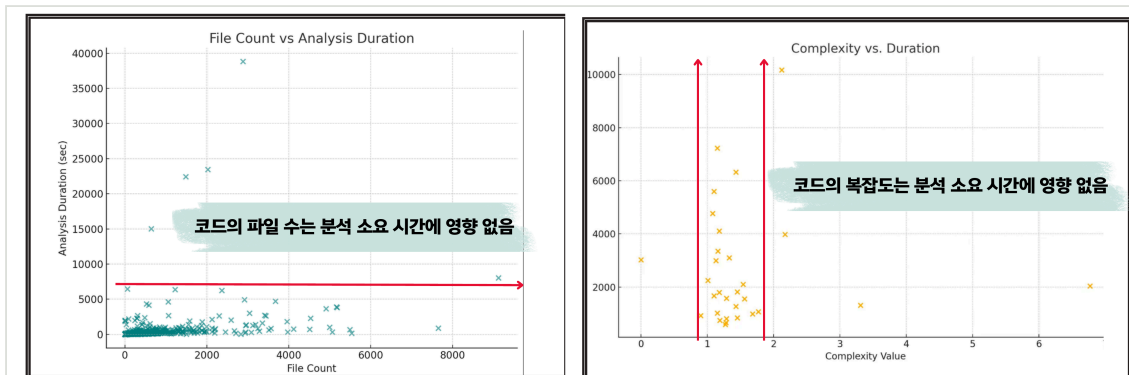


그림 1. 파일 수와 평균 복잡도는 분석 소요 시간과 뚜렷한 관계를 보이지 않았다.

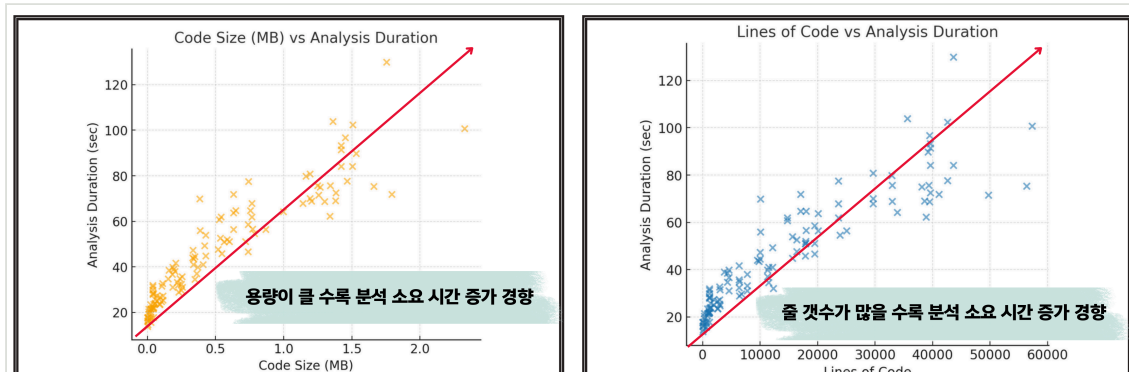


그림 2. 반면 코드 용량(MB)과 전체 줄 수는 분석 소요 시간과 함께 거의 선형적으로 증가했다.

요약하면 파일 수와 평균 복잡도는 영향이 거의 없었고, 코드 용량과 줄 수가 분석 시간을 좌우했다. 이 두 값을 핵심 입력 변수로 삼았다.

5. 모델링: 선형 회귀에서 XGBoost로

처음에는 용량과 줄 수에 대한 언어별 선형 회귀로 접근했다. 언어별로는 큰 추세를 잡아냈지만(Java R^2 0.93, Python 0.93) 단일 직선으로는 요인 간 복합적 상호작용을 표현하지 못해 예측이 불안정했다.

이에 28개 피처의 상호작용을 학습할 수 있는 **XGBoost 그라디언트 부스팅**으로 전환했다. 타깃은 로그 변환한 분석 소요 시간을 사용했고, 예측값은 다시 초 단위로 역변환했다.

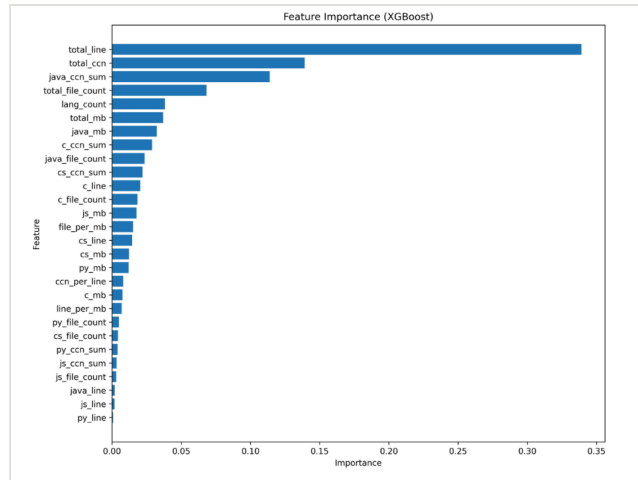


그림 3. 피처 중요도. **total_line**(전체 줄 수)이 가장 큰 영향을 미쳤고, 그다음이 **total_ccn**(전체 복잡도 합)이었다. 선형 모델이 놓쳤던 비선형 상호작용을 모델이 포착했다.

```
# 모델 구성
XGBRegressor(n_estimators=100, max_depth=5)
# target: 로그 변환 duration · features: 28 · split: train 90% / test 10% · metric: R2
y_pred = np.expml(model.predict(X)) # → 소요 시간(초)
```

6. 결과

XGBoost로 학습한 모델의 결정계수 R^2 은 평균 **0.8963**(최저 0.7321, 최고 0.9610, 표준편차 0.0392)을 기록했다. 선형 회귀로는 요인 간 상호작용을 충분히 반영하지 못해 예측이 불안정했지만, 28개 피처를 함께 학습한 XGBoost는 안정적으로 높은 설명력을 보였다.

표 3. XGBoost 예측 결과 표본 (실제 vs 예측).

분석 ID	실제 소요 시간(초)	모델 예측 시간(초)
10039	84	86.34
10038	76	76.02
10037	62	66.66
10036	69	78.57
10035	64	89.86
10033	104	65.02
10032	102	91.55
10031	90	83.68
10030	93	81.40
10029	97	83.72
10022	1017	940.32
10021	764	727.84

표 4. 언어별 선형 회귀 R² (참고).

언어	R ²
Java	0.93
Python	0.93
C / C++	0.83
C#	0.79
JavaScript / TS	0.70

이상 탐지 기준. 실제 소요 시간이 모델 예측값의 $\pm 20\%$ 범위 안에 들면 정상, 벗어나면 이상으로 판정한다.

7. 서비스 연동

탐지 모델을 정적 분석 서비스에 연동하고, 스스로 개선되는 재학습 루프를 구성했다. GitHub Actions가 일정 주기로 PostgreSQL에서 최신 분석 데이터를 조회하고, 축적된 데이터로 모델을 다시 학습해 정확도를 지속적으로 끌어올린다.

새 분석의 실제 시간이 예측 $\pm 20\%$ 범위를 벗어나면 Discord 봇(Captain Hook)이 분석 링크와 함께 예측·실제 시간을 담은 알림을 보낸다. 이 기능은 **SPARROW 2508.1 릴리스**에 적용 예정으로 개발했다.

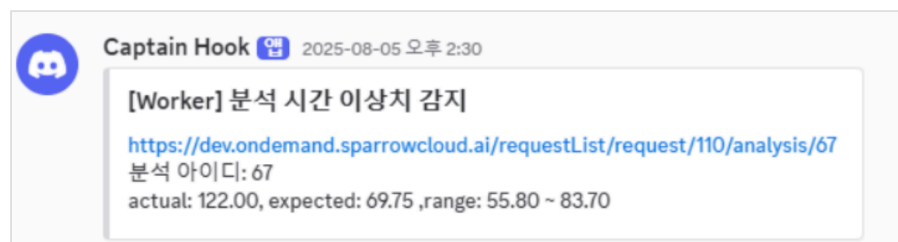


그림 4. 실제 이상 알림 예시. 예측 69.75초(정상 범위 55.80~83.70) 대비 실제 122.00초로 이상이 탐지되었고, 분석 페이지로 바로 이동하는 링크가 포함된다.

8. 결론과 향후 과제

도입 전에는 이상 분석을 자동으로 알 방법이 없어 누군가 직접 확인하기 전까지 방치되었고, 개발자는 멈춘 듯한 분석을 기다리느라 시간을 낭비했다. 도입 후에는 비정상적으로 오래 걸리는 분석을 자동으로, 실시간에 가깝게 탐지해 알리며, 모델은 데이터가 쌓일수록 스스로 정확도를 높인다.

향후 과제

- 대용량 프로젝트 중심으로 데이터를 추가 수집해 정확도를 더 끌어올린다.
- 현재 5개 지원 언어 외 추가 언어로 탐지 범위를 확장한다.
- 클라우드 환경에 배포해 운영 안정성을 높인다.